

Занятие 5. Диффузия и score: тот же transport landscape, но не «то же самое» без условий

Какой вопрос закрывает занятие

На занятии 4 генерация была описана как transport problem. Есть probability path p_τ , есть vector field $v_\tau(x)$, который двигает probability mass вдоль этого path, а sampling сводится к интегрированию ODE. Conditional Flow Matching позволял обучать такое поле обычной regression на доступные conditional velocities.

Теперь возникает естественный вопрос:

где в этой картине находится diffusion, зачем ей нужен score и действительно ли diffusion и flow matching описывают одно и то же?

Короткий ответ такой: связь очень тесная, но фраза «это одно и то же» без дополнительных условий неверна.

Diffusion задаёт probability path через постепенное зашумление данных. Для Gaussian corruption intermediate sample строится как

$$x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Такой path можно описывать несколькими способами. Можно предсказывать noise ϵ . Можно восстанавливать clean sample x_0 . Можно учить score $s_\tau(x) = \nabla_x \log p_\tau(x)$.

Можно перейти к velocity target. Наконец, если Gaussian path порождён diffusion SDE, тот же family of marginals допускает deterministic probability-flow ODE.

То есть diffusion, score models и flow matching действительно находятся в одном **transport landscape**. Но здесь важно не перепутать несколько разных утверждений:

- одни и те же training targets могут быть алгебраически преобразуемы друг в друга;
- соответствующие идеальные marginal fields могут описывать один probability path;
- две MSE objectives могут совпадать после правильного преобразования весов;
- две реально обученные finite-capacity networks могут получиться разными.

Это четыре разных уровня утверждения. Переход с одного уровня на другой каждый раз требует отдельного аргумента.

Есть одно явное исключение из orientation занятия 4. Символ $\tau \in [0, 1]$ по-прежнему обозначает continuous generative/noise time, $v_\tau(x)$ — marginal vector field, а u_τ — conditional velocity, но направление increasing τ ниже меняется.

Важно: смена orientation τ . На занятии 4 increasing τ шло от source к data. В этом занятии increasing τ означает forward noising от clean data к noise, а generation идёт при decreasing τ от noise к data. Символ времени остаётся тем же, но orientation нужно учитывать явно. Clean sample в Gaussian corruption поэтому обозначается x_0 ; token position по-прежнему обозначается t .

Что нужно знать заранее

Предполагается, что после занятия 4 уже понятны:

- probability path p_τ ;
- difference между individual trajectory и marginal probability path;
- vector field $v_\tau(x)$;
- conditional velocity u_τ ;
- Conditional Flow Matching;
- факт, что MSE regression на random target U восстанавливает conditional mean $\mathbb{E}[U \mid Z]$;
- ODE sampling;
- solver step и NFE;
- различие между mathematical operator, architecture и algorithm его вычисления.

Теория stochastic differential equations не предполагается. SDE понадобится только как язык для двух результатов: reverse-time SDE и probability-flow ODE.

Что должно получаться после чтения

После этого конспекта нужно уметь без дополнительных материалов:

1. Из $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ вывести conditional distribution $p(x_\tau \mid x_0)$.
2. Без подсказки дать правильное определение score $s_\tau(x) = \nabla_x \log p_\tau(x)$ и объяснить, почему логарифм здесь принципиален.
3. Полностью вывести conditional Gaussian score и получить target $-\epsilon/\sigma_\tau$.
4. Объяснить denoising identity: почему regression на conditional score targets восстанавливает marginal score.
5. Перевести ϵ -prediction в score prediction и x_0 -prediction.

6. Для normalized path восстановить преобразования между x_τ , x_0 , ϵ и velocity-prediction target.
7. Объяснить, почему velocity-prediction target v_τ^{VP} и ODE vector field $v_\tau(x)$ - разные объекты.
8. Для заданных α_τ , σ_τ получить conditional transport velocity u_τ .
9. Объяснить, как score определяет probability-flow ODE.
10. Сформулировать условия, при которых diffusion и flow-matching formulations действительно можно переводить друг в друга.
11. Показать, где при таком переходе возникают time-dependent loss weights.
12. Объяснить, почему DiT меняет backbone, но не саму diffusion probability construction.
13. Посчитать реальную работу vanilla classifier-free guidance, не смешивая sampler steps, network calls и обработанный объём.
14. Отдельно назвать NFE, latency, throughput, processed elements и peak memory и объяснить, почему ни одна из этих величин не заменяет остальные.
15. Объяснить, почему утверждение «у diffusion models мало памяти» неверно как утверждение о целом классе моделей.

Словарь занятия

Понятие	English term	Определение
Прямой процесс зашумления	forward noising process	Процесс, который постепенно превращает data sample в всё более шумный sample. Generation использует обратное направление.
Gaussian corruption	Gaussian corruption	Зашумление вида $\boldsymbol{x}_\tau = \alpha_\tau \boldsymbol{x}_0 + \sigma_\tau \epsilon$, где ϵ имеет standard Gaussian distribution.
Noise schedule	noise schedule	Правило, задающее зависимость степени corruption от τ , в частности functions α_τ и σ_τ .
Signal coefficient	signal coefficient	α_τ - коэффициент, с которым clean signal $\boldsymbol{x}_0$ входит в $\boldsymbol{x}_\tau$.
Noise coefficient	noise coefficient	σ_τ - коэффициент standard Gaussian noise. Conditional variance равна $\sigma_\tau^2 I$.
Score	score function	Gradient log-density: $\boldsymbol{s}_\tau(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \log p_\tau(\boldsymbol{x})$.
Conditional score	conditional score	$\nabla_{\boldsymbol{x}_\tau} \log p(\boldsymbol{x}_\tau \mid \boldsymbol{x}_0)$ - score conditional corruption kernel при известном clean sample.
Marginal score	marginal score	$\nabla_{\boldsymbol{x}} \log p_\tau(\boldsymbol{x})$, где clean sample уже marginalize out. Именно этот объект нужен generative dynamics.
Denoising score matching	denoising score matching	Training construction, при котором model регрессирует на доступный conditional score, а conditional expectation этого target равен marginal score.
Noise prediction	ϵ -prediction	Parameterization, в которой model по $(\boldsymbol{x}_\tau, \tau)$ предсказывает noise ϵ .
Clean-data prediction	$\boldsymbol{x}_0$ -prediction	Parameterization, в которой model предсказывает clean sample.
Score prediction	score prediction	Parameterization, в которой network output непосредственно интерпретируется как $\boldsymbol{s}_\tau(\boldsymbol{x})$.
Velocity prediction	$\boldsymbol{v}$ -prediction	Diffusion parameterization с target $\boldsymbol{v}_\tau^{\text{VP}} = \alpha_\tau \epsilon - \sigma_\tau \boldsymbol{x}_0$ для normalized schedule. $\boldsymbol{v}_\tau^{\text{VP}}$ не совпадает автоматически с ODE vector field $\boldsymbol{v}_\tau(\boldsymbol{x})$.

Понятие	English term	Определение
Stochastic differential equation	SDE	Continuous-time dynamics с deterministic drift и stochastic Brownian increment.
Reverse-time SDE	reverse-time SDE	SDE, которая при известном score позволяет идти от noisy terminal distribution обратно к data distribution.
Probability-flow ODE	probability-flow ODE	Deterministic ODE, имеющая те же time marginals, что соответствующая diffusion SDE.
Diffusion path	diffusion probability path	Probability path, возникающий из постепенного stochastic noising process.
Diffusion Transformer	DiT	Transformer backbone для diffusion model, в исходной работе работающий над latent patches.
Classifier-Free Guidance	CFG	Guidance scheme, комбинирующая conditional и unconditional predictions одной generative model.
Sampler step	sampler step	Один macro-step numerical sampling algorithm. Может требовать несколько network evaluations.
NFE	number of function evaluations	Число evaluations learned function. При guidance нужно явно уточнять, считаются ли внутренние conditional/unconditional branches отдельно.
Latency	latency	Wall-clock время получения результата при зафиксированных hardware, batch и implementation.
Throughput	throughput	Число результатов или обработанных элементов за единицу времени при заданной concurrency.
Peak memory	peak memory	Максимальный одновременно занятый объём памяти во время computation. Не равен сумме памяти по всем sampling steps.

1. Gaussian corruption

Зачем эта тема здесь

На занятии 4 probability path выбирался напрямую: например, через interpolation между source и target samples.

Diffusion делает другой design choice. Вместо того чтобы сразу сказать, как двигать data point к noise или noise к data, сначала задаётся очень простой **forward corruption process**: data постепенно смешиваются с Gaussian noise.

Преимущество такой construction в том, что conditional distribution intermediate state известна аналитически. Значит, training examples на произвольном noise level можно получать без numerical simulation всей trajectory.

Интуиция

Возьмём clean sample x_0 .

На маленьком noise level хочется почти сохранить его: $x_\tau \approx x_0$.

На большом noise level хочется почти забыть его и получить Gaussian noise: $x_\tau \approx \epsilon$.

Самый простой способ интерполировать между этими режимами: $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$.

α_τ отвечает за количество signal, σ_τ - за количество noise.

Для типичного data-to-noise schedule:

$$\begin{aligned} \alpha_0 &\approx 1, & \sigma_0 &\approx 0, \\ \alpha_1 &\approx 0, & \sigma_1 &\approx 1. \end{aligned}$$

Знак $\approx$ здесь осознанный. Некоторые constructions достигают точных endpoints, некоторые только практически очень близки к ним.

Понятия, которые вводятся

Нужны forward noising process, Gaussian corruption, noise schedule, signal coefficient и noise coefficient.

Формальное изложение

Пусть clean sample фиксирован: $x_0 \in \mathbb{R}^d$.

Берём independent standard Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$.

Определяем $X_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$.

Здесь randomness содержится только в ϵ .

Нужно получить distribution X_τ при фиксированном x_0 .

Шаг 1. Mean

По linearity expectation:

$$\begin{aligned}\mathbb{E}[X_\tau \mid x_0] &= \alpha_\tau x_0 + \sigma_\tau \mathbb{E}[\epsilon] \\ &= \alpha_\tau x_0,\end{aligned}$$

поскольку $\mathbb{E}[\epsilon] = 0$.

Шаг 2. Covariance

Вычтем conditional mean:

$$X_\tau - \mathbb{E}[X_\tau \mid x_0] = \sigma_\tau \epsilon.$$

Следовательно,

$$\begin{aligned}\text{Cov}(X_\tau \mid x_0) &= \sigma_\tau^2 \text{Cov}(\epsilon) \\ &= \sigma_\tau^2 I.\end{aligned}$$

Шаг 3. Distribution

Affine transformation Gaussian vector снова Gaussian.

Поэтому

$$p(x_\tau | x_0) = \mathcal{N}(\alpha_\tau x_0, \sigma_\tau^2 I).$$

То есть σ_τ - это **standard-deviation coefficient**, а variance содержит квадрат σ_τ^2 .

При $\sigma_\tau = 0$ distribution вырождается в point mass. Поэтому formulas с ordinary Gaussian density и score дальше понимаются для interior noise levels, где $\sigma_\tau > 0$.

Marginal noisy distribution

Clean sample сам случаен: $X_0 \sim p_{\text{data}}$.

Если X_0 скрыть, получается marginal distribution noisy samples:

$$p_\tau(x) = \int p_{\text{data}}(x_0) p(x | x_0) dx_0.$$

Вот это p_τ и есть один element probability path.

Важно различать два объекта: $p(x_\tau | x_0)$ известен аналитически, а $p_\tau(x)$ обычно является сложной смесью Gaussian kernels и аналитически не известен.

Именно поэтому conditional training target получить легко, а marginal score напрямую посчитать трудно.

Normalized schedules

Часто используется convention $\alpha_\tau^2 + \sigma_\tau^2 = 1$.

В этом случае total scale signal и noise нормирован.

Но это **не определение diffusion вообще**. Существуют и другие parameterizations, поэтому равенство $\alpha_\tau^2 + \sigma_\tau^2 = 1$ будем использовать только там, где оно явно предположено.

Где здесь DDPM

DDPM Ho, Jain, Abbeel - discrete diffusion model 2020 года. Forward process там является Markov chain, постепенно добавляющей Gaussian noise.

В оригинальной diffusion notation шаги обычно обозначаются $t = 1, \dots, T$. В этом курсе t уже занят token position, поэтому discrete diffusion index обозначим ℓ .

Один DDPM forward transition имеет вид

$$q(x_\ell | x_{\ell-1}) = \mathcal{N}(\sqrt{1 - \beta_\ell} x_{\ell-1}, \beta_\ell I).$$

Если определить

$$a_\ell = 1 - \beta_\ell, \quad \bar{a}_\ell = \prod_{j=1}^{\ell} a_j,$$

то cumulative corruption можно записать сразу:

$$q(x_\ell | x_0) = \mathcal{N}(\sqrt{\bar{a}_\ell} x_0, (1 - \bar{a}_\ell)I).$$

Это тот же structural pattern:

$$\alpha_\ell = \sqrt{\bar{a}_\ell}, \quad \sigma_\ell = \sqrt{1 - \bar{a}_\ell}.$$

То есть continuous notation $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ удобно воспринимать как continuous analogue той же Gaussian corruption idea.

Полный variational ELBO DDPM, derivation reverse posterior $q(x_{\ell-1} | x_\ell, x_0)$ и устройство discrete ancestral sampler здесь **не разбираются**. Они нужны, если цель - подробно изучить именно DDPM как latent-variable model. Здесь нас интересует связь diffusion с score и transport.

Пример

Возьмём normalized trigonometric schedule

$$\alpha_\tau = \cos \frac{\pi\tau}{2}, \quad \sigma_\tau = \sin \frac{\pi\tau}{2}.$$

При $\tau = \frac{1}{2}$:

$$\alpha_\tau = \sigma_\tau = \frac{1}{\sqrt{2}}.$$

Пусть в одномерном примере $x_0 = 2$, $\epsilon = -1$.

Тогда

$$x_\tau = \frac{2}{\sqrt{2}} - \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{2}}.$$

Conditional distribution в этот момент имеет mean $\alpha_\tau x_0 = \sqrt{2}$ и variance

$$\sigma_{\tau}^2 = \frac{1}{2}.$$

Sample оказался левее conditional mean, потому что в него попал negative noise.

Где здесь легко ошибиться

« σ_{τ} - это variance»

Нет.

В construction $x_{\tau} = \alpha_{\tau}x_0 + \sigma_{\tau}\epsilon$ noise covariance равна $\sigma_{\tau}^2 I$.

«Forward noising и generation идут в одну сторону»

Нет.

Forward process удобен для определения training distribution: **data** $\rightarrow$ **noise**.

Generation решает обратную задачу: **noise** $\rightarrow$ **data**.

«Если conditional distribution Gaussian, то p_{τ} тоже Gaussian»

Вообще говоря, нет.

p_{τ} является mixture по data distribution. Если data distribution сложная, noisy marginal тоже может оставаться сложным, особенно при небольшом noise level.

«Любой schedule обязан удовлетворять $\alpha_{\tau}^2 + \sigma_{\tau}^2 = 1$ »

Нет.

Это важный и распространённый normalized convention, но не общий definition.

«DDPM - это просто одна формула $x_{\tau} = \alpha_{\tau}x_0 + \sigma_{\tau}\epsilon$ »

Не совсем.

DDPM задаёт discrete Markov forward process и learned reverse model. Closed-form corruption from x_0 to arbitrary noise level - очень удобное следствие этой construction, но не полное определение всей model.

Что теперь понятно

Gaussian corruption задаёт tractable conditional path от clean sample к noise.

Мы умеем:

- sampled arbitrary noisy state без simulation промежуточных steps;
- точно записать $p(x_\tau \mid x_0)$;
- разделить известный conditional kernel и неизвестный noisy marginal p_τ .

Следующий вопрос - как получить из известного conditional Gaussian информацию о неизвестном marginal distribution.

Для этого появляется score.

2. Score и denoising target

Зачем эта тема здесь

Это центральный раздел занятия.

Forward corruption известен аналитически, но generative dynamics должна работать без знания clean x_0 . Во время sampling model видит только текущий noisy state x и noise level τ .

Нужен объект, который описывает local geometry marginal distribution $p_\tau(x)$.

Этим объектом является score.

И здесь важно исправить конкретную ошибку: score - **не gradient density**.

Правильное определение:

$$s_\tau(x) = \nabla_x \log p_\tau(x).$$

Интуиция

В одном измерении derivative $\frac{d}{dx} \log p(x)$ отвечает на вопрос:

насколько быстро density меняется относительно своего текущего значения?

Поскольку

$$\nabla_x \log p(x) = \frac{\nabla_x p(x)}{p(x)},$$

score является relative slope, а не absolute slope density.

Это distinction особенно заметно в tails.

Для standard Gaussian:

$$p(x) \propto \exp\left(-\frac{x^2}{2}\right).$$

Score: $\nabla_x \log p(x) = -x$.

Gradient самой density: $\nabla_x p(x) = -xp(x)$.

При большом x density $p(x)$ очень мала, поэтому $\nabla p(x)$ тоже почти нулевая.

Но score остаётся большим по magnitude и указывает обратно к high-density region.

То есть gradient density говорит в том числе о **масштабе самой density**, а score выделяет её local relative geometry.

Почему логарифм принципиален

Есть три причины, и все содержательные.

1. Score не зависит от normalization constant

Пусть известна unnormalized density $\tilde{p}(x) = C p(x)$, где C не зависит от x .

Тогда

$$\begin{aligned}\nabla_x \log \tilde{p}(x) &= \nabla_x (\log C + \log p(x)) \\ &= \nabla_x \log p(x).\end{aligned}$$

Normalization constant исчезает.

У gradient density такого свойства нет: $\nabla_x \tilde{p}(x) = C \nabla_x p(x)$.

Это одна из причин, почему log-density gradients вообще так удобны в probabilistic modeling.

2. Score измеряет relative change

Из

$$s(x) = \frac{\nabla p(x)}{p(x)}$$

видно, что одна и та же absolute derivative имеет разный смысл в high-density и low-density regions.

Score автоматически учитывает эту нормировку.

3. Именно log-gradient даёт denoising identity

Это особенно важно для нашего курса.

Позже получится:

$$\nabla_x \log p_\tau(x) = \mathbb{E} [\nabla_x \log p(x | X_0) | X_\tau = x].$$

В этой formula conditional density внутри gradient сокращается с posterior weight.

Если вместо $\nabla \log p(x | x_0)$ взять просто $\nabla p(x | x_0)$, такого cancellation не получится.

То есть логарифм здесь не cosmetic notation. Без него ломается сам простой bridge от conditional Gaussian target к marginal field.

Понятия, которые вводятся

Нужны score function, conditional score, marginal score, denoising score matching и denoising identity.

Формальное изложение

Conditional Gaussian density

Из предыдущего раздела:

$$p(x_\tau \mid x_0) = \mathcal{N}(\alpha_\tau x_0, \sigma_\tau^2 I).$$

Для $\sigma_\tau > 0$ density можно записать как

$$p(x_\tau \mid x_0) = \frac{1}{(2\pi\sigma_\tau^2)^{d/2}} \exp\left(-\frac{\|x_\tau - \alpha_\tau x_0\|_2^2}{2\sigma_\tau^2}\right).$$

Берём logarithm:

$$\begin{aligned} \log p(x_\tau \mid x_0) &= -\frac{d}{2} \log(2\pi\sigma_\tau^2) \\ &\quad - \frac{1}{2\sigma_\tau^2} \|x_\tau - \alpha_\tau x_0\|_2^2. \end{aligned}$$

Первое term не зависит от x_τ .

Поэтому при differentiation по x_τ оно исчезает.

Остаётся

$$\nabla_{x_\tau} \log p(x_\tau \mid x_0) = -\frac{1}{2\sigma_\tau^2} \nabla_{x_\tau} \|x_\tau - \alpha_\tau x_0\|_2^2.$$

Для vector z : $\nabla_z \|z\|_2^2 = 2z$.

Здесь $z = x_\tau - \alpha_\tau x_0$.

Следовательно,

$$\begin{aligned} &\nabla_{x_\tau} \log p(x_\tau \mid x_0) \\ &= -\frac{1}{2\sigma_\tau^2} 2(x_\tau - \alpha_\tau x_0). \end{aligned}$$

Итого:

$$\boxed{\nabla_{x_\tau} \log p(x_\tau \mid x_0) = -\frac{x_\tau - \alpha_\tau x_0}{\sigma_\tau^2}.$$

Теперь используем reparameterization $x_\tau - \alpha_\tau x_0 = \sigma_\tau \epsilon$.

Получаем

$$\nabla_{x_\tau} \log p(x_\tau | x_0) = -\frac{\epsilon}{\sigma_\tau}.$$

Это основной training target, который связывает diffusion noise prediction и score prediction.

Но model нужен marginal score

Полученный объект всё ещё зависит от hidden clean sample: $\nabla_x \log p(x | x_0)$.

Во время generation x_0 неизвестен.

Нужен $s_\tau(x) = \nabla_x \log p_\tau(x)$.

Почему regression на первый объект восстанавливает второй?

Здесь повторяется ровно та regression logic, которая была нужна в Conditional Flow Matching.

Denoising identity

Marginal density:

$$p_\tau(x) = \int p_{\text{data}}(x_0) p(x | x_0) dx_0.$$

При обычных regularity assumptions можно перенести gradient под integral:

$$\nabla_x p_\tau(x) = \int p_{\text{data}}(x_0) \nabla_x p(x | x_0) dx_0.$$

Теперь используем identity

$$\nabla_x p(x | x_0) = p(x | x_0) \nabla_x \log p(x | x_0).$$

Тогда

$$\begin{aligned} \nabla_x p_\tau(x) &= \int p_{\text{data}}(x_0) p(x | x_0) \\ &\quad \nabla_x \log p(x | x_0) dx_0. \end{aligned}$$

Делим на $p_\tau(x)$:

$$\nabla_x \log p_\tau(x) = \int \frac{p_{\text{data}}(x_0)p(x | x_0)}{p_\tau(x)} \nabla_x \log p(x | x_0) dx_0.$$

Но по Bayes rule коэффициент

$$\frac{p_{\text{data}}(x_0)p(x | x_0)}{p_\tau(x)}$$

есть posterior density clean sample при наблюдаемом noisy x .

Поэтому

$$s_\tau(x) = \mathbb{E} [\nabla_x \log p(X_\tau = x | X_0) | X_\tau = x].$$

Для Gaussian corruption:

$$s_\tau(x) = -\frac{1}{\sigma_\tau} \mathbb{E}[\epsilon | X_\tau = x].$$

Полное measure-theoretic proof нам не нужно. Содержательное statement именно такое:

marginal score в point x равен posterior average conditional scores всех clean/noise configurations, которые могли породить наблюдаемый x .

Это почти дословный аналог Conditional Flow Matching:

$$\text{hidden conditional target} \rightarrow \mathbb{E}[\text{target} | \text{observed state}].$$

Почему MSE regression работает

Пусть network предсказывает noise: $\hat{\epsilon}_\theta(x_\tau, \tau)$.

Обучаем

$$\mathcal{L}_\epsilon = \mathbb{E} [w_\epsilon(\tau) \|\hat{\epsilon}_\theta(X_\tau, \tau) - \epsilon\|_2^2].$$

Если сначала зафиксировать τ и $X_\tau = x$, optimal squared-error predictor:

$$\hat{\epsilon}^*(x, \tau) = \mathbb{E}[\epsilon | X_\tau = x, \tau].$$

Следовательно,

$$s_{\tau}(x) = -\frac{\hat{\epsilon}^*(x, \tau)}{\sigma_{\tau}}.$$

Model не обязана угадывать конкретный noise realization конкретной training pair.

Она восстанавливает posterior mean noise.

Это distinction такое же, как в CFM: deterministic model видит только (x, τ) и возвращает average target среди hidden situations compatible с этим input.

Пример: conditional targets конфликтуют, marginal score один

Пусть в одном измерении clean data принимает два значения с одинаковой probability: $X_0 \in \{-1, 1\}$.

Возьмём $\alpha_{\tau} = 1$, $\sigma_{\tau} = 1$.

Рассмотрим observed noisy point $X_{\tau} = 0$.

Если hidden clean sample был $X_0 = 1$, то $\epsilon = -1$ и conditional score target: $-\epsilon = 1$.

Если hidden clean sample был $X_0 = -1$, то $\epsilon = 1$ и target: $-\epsilon = -1$.

Из symmetry posterior probabilities двух cases одинаковы.

Поэтому marginal score в $x = 0$:

$$s_{\tau}(0) = \frac{1}{2}(1) + \frac{1}{2}(-1) = 0.$$

Network не выбирает один из hidden clean samples.

Она возвращает single-valued marginal quantity.

Где здесь легко ошибиться

«Score - это gradient density»

Нет.

Правильная связь: $\nabla_x p_{\tau}(x) = p_{\tau}(x) s_{\tau}(x)$.

Gradient density и score могут указывать в одно direction, если density positive, но имеют совершенно разный scale и разные algebraic properties.

В частности, denoising identity использует именно log-gradient.

« $-\epsilon/\sigma_\tau$ уже является marginal score каждого training sample»

Нет.

Это conditional score target.

Marginal score получается после conditional averaging по hidden clean/noise configurations.

«Model должна восстановить именно тот noise, который использовали при corruption»

Не обязана и часто не может.

Один observed x_τ может быть совместим со многими (x_0, ϵ) .

MSE optimum - conditional expectation.

«Score хорошо определён и в exact clean endpoint $\sigma_\tau = 0$ »

Формула $-\epsilon/\sigma_\tau$ там singular.

Практические objectives либо не sample exact endpoint, либо используют weighting/parameterization, при которых numerical behaviour контролируется.

«Log нужен только потому, что так принято в papers»

Нет.

Без log теряется normalization invariance и, главное для этой лекции, перестаёт работать простая conditional-to-marginal identity.

Что теперь понятно

Из Gaussian corruption можно получить точный conditional score.

MSE regression на этот conditional target восстанавливает marginal score точно по той же conditional-expectation logic, которая уже встречалась в Flow Matching.

Получилась первая серьёзная связь двух занятий:

CFM: conditional velocity $\rightarrow$ marginal velocity,

DSM: conditional score $\rightarrow$ marginal score.

Но velocity и score пока всё ещё разные mathematical objects.

Чтобы связать их, сначала нужно разобраться с parameterizations.

3. Четыре parameterizations: ϵ , x_0 , score и velocity

Зачем эта тема здесь

В diffusion papers можно встретить внешне разные training targets:

- predict noise;
- predict clean data;
- predict score;
- predict velocity.

Из-за этого легко решить, что речь идёт о четырёх принципиально разных generative models.

На sample level это слишком сильный вывод: для фиксированного Gaussian path targets часто алгебраически переводятся друг в друга.

Но отсюда очень легко сделать следующую, уже противоположную ошибку:

раз targets переводятся друг в друга, значит training objectives и обученные networks эквивалентны.

Это тоже неверно без условий.

Интуиция

Пусть известны x_τ и time τ .

Equation $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ содержит две hidden quantities: x_0 и ϵ .

Если model дала estimate одной из них, вторую можно восстановить алгебраически.

Score тоже связан с posterior mean noise.

Velocity-prediction target при normalized schedule является ещё одной linear combination тех же x_0 и ϵ .

На уровне ideal values это просто смена coordinates.

Но MSE измеряет error в выбранных coordinates. Если coordinate transformation зависит от τ , то одинаковый physical error получает разные weights в разных parts probability path.

Вот здесь и возникает отличие training objectives.

Понятия, которые вводятся

Нужны target parameterization, noise prediction, clean-data prediction, score prediction, velocity prediction и loss weighting.

Формальное изложение

Из ϵ в score

Conditional score:

$$s_\tau^{\text{cond}} = -\frac{\epsilon}{\sigma_\tau}.$$

Поэтому noise predictor естественно превращается в score estimate:

$$\hat{s}_\tau(x_\tau) = -\frac{\hat{\epsilon}_\theta(x_\tau, \tau)}{\sigma_\tau}.$$

При exact MSE population optimum это даёт marginal score, поскольку $\hat{\epsilon}^* = \mathbb{E}[\epsilon \mid X_\tau]$.

Обратное преобразование:

$$\hat{\epsilon}_\theta = -\sigma_\tau \hat{s}_\tau.$$

Из ϵ в x_0

Из $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ при $\alpha_\tau \neq 0$: $\alpha_\tau x_0 = x_\tau - \sigma_\tau \epsilon$.

Поэтому

$$\hat{x}_0 = \frac{x_\tau - \sigma_\tau \hat{\epsilon}_\theta}{\alpha_\tau}.$$

Если вместо noise predictor подставить score predictor: $\hat{\epsilon}_\theta = -\sigma_\tau \hat{s}_\tau$, то

$$\hat{x}_0 = \frac{x_\tau + \sigma_\tau^2 \hat{s}_\tau}{\alpha_\tau}.$$

При exact marginal score эта formula возвращает posterior mean clean sample, а не magic reconstruction скрытого training x_0 из информации, которой в x_τ уже нет.

Velocity-prediction parameterization

Теперь дополнительно предположим normalized path: $\alpha_\tau^2 + \sigma_\tau^2 = 1$.

Вводится target

$$v_\tau^{\text{VP}} = \alpha_\tau \epsilon - \sigma_\tau x_0.$$

Здесь superscript нужен специально, чтобы не смешивать этот объект с marginal ODE vector field $v_\tau(x)$ занятия 4.

Рассмотрим pair equations:

$$\begin{aligned} x_\tau &= \alpha_\tau x_0 + \sigma_\tau \epsilon, \\ v_\tau^{\text{VP}} &= -\sigma_\tau x_0 + \alpha_\tau \epsilon. \end{aligned}$$

Это rotation в plane, образованной x_0 и ϵ .

Из-за normalization inverse transformation особенно простой:

$$x_0 = \alpha_\tau x_\tau - \sigma_\tau v_\tau^{\text{VP}}.$$

Проверим.

Подставляем definitions:

$$\begin{aligned}
& \alpha_\tau x_\tau - \sigma_\tau v_\tau^{\text{VP}} \\
&= \alpha_\tau(\alpha_\tau x_0 + \sigma_\tau \epsilon) - \sigma_\tau(\alpha_\tau \epsilon - \sigma_\tau x_0) \\
&= (\alpha_\tau^2 + \sigma_\tau^2)x_0 = x_0.
\end{aligned}$$

А для noise:

$$\epsilon = \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}}.$$

Проверка:

$$\begin{aligned}
& \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}} \\
&= \sigma_\tau(\alpha_\tau x_0 + \sigma_\tau \epsilon) + \alpha_\tau(\alpha_\tau \epsilon - \sigma_\tau x_0) \\
&= (\alpha_\tau^2 + \sigma_\tau^2)\epsilon = \epsilon.
\end{aligned}$$

Значит, при normalized parameterization:

$$\begin{aligned}
x_0 &= \alpha_\tau x_\tau - \sigma_\tau v_\tau^{\text{VP}}, \\
\epsilon &= \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}}.
\end{aligned}$$

Почему этот v не равен vector field занятия 4

Это место лучше разобрать отдельно.

В Flow Matching conditional path $X_\tau = \alpha_\tau X_0 + \sigma_\tau \epsilon$ имеет conditional velocity

$$U_\tau = \dot{\alpha}_\tau X_0 + \dot{\sigma}_\tau \epsilon.$$

Velocity-prediction target:

$$v_\tau^{\text{VP}} = \alpha_\tau \epsilon - \sigma_\tau X_0.$$

Это вообще говоря разные linear combinations X_0 и ϵ .

Для normalized schedule удобно записать

$$\alpha_\tau = \cos \phi_\tau, \quad \sigma_\tau = \sin \phi_\tau.$$

Тогда

$$\dot{\alpha}_\tau = -\dot{\phi}_\tau \sigma_\tau, \quad \dot{\sigma}_\tau = \dot{\phi}_\tau \alpha_\tau.$$

Подставляем в conditional transport velocity:

$$\begin{aligned}
 U_\tau &= -\dot{\phi}_\tau \sigma_\tau X_0 + \dot{\phi}_\tau \alpha_\tau \epsilon \\
 &= \dot{\phi}_\tau (\alpha_\tau \epsilon - \sigma_\tau X_0).
 \end{aligned}$$

Следовательно,

$$U_\tau = \dot{\phi}_\tau v_\tau^{\text{VP}}.$$

Уже отсюда видно: даже conditional transport velocity и diffusion v -target совпадают только при конкретном choice time parameterization.

Например, если

$$\phi_\tau = \frac{\pi\tau}{2},$$

то

$$U_\tau = \frac{\pi}{2} v_\tau^{\text{VP}}.$$

Они proportional, но не равны.

И это ещё не весь distinction.

U_τ - random **conditional training velocity**, зависящая от hidden pair (X_0, ϵ) .

Vector field занятия 4: $v_\tau(x) = \mathbb{E}[U_\tau \mid X_\tau = x]$.

Это уже marginal deterministic function текущего x и τ .

Поэтому одновременно различаем три объекта:

$$\begin{aligned}
 v_\tau^{\text{VP}} &\quad \text{- diffusion targetparameterization,} \\
 U_\tau &\quad \text{- conditional path velocity,} \\
 v_\tau(x) &\quad \text{- marginal ODE vector field.}
 \end{aligned}$$

Совпадение letter v в литературе не делает их одним object.

Алгебраическая связь не означает одинаковую training objective

Теперь центральный вопрос.

Пусть $e_\epsilon = \hat{\epsilon} - \epsilon$.

Score error

Поскольку

$$\hat{s} - s^{\text{cond}} = -\frac{e_\epsilon}{\sigma_\tau},$$

то

$$\|\hat{s} - s^{\text{cond}}\|_2^2 = \frac{1}{\sigma_\tau^2} \|e_\epsilon\|_2^2.$$

Unweighted score MSE поэтому соответствует ϵ -error, дополнительно умноженной на $1/\sigma_\tau^2$.

Чтобы weighted score loss буквально совпала с unweighted ϵ -loss, нужно взять weight $w_s(\tau) = \sigma_\tau^2$.

x_0 error

Из reconstruction formula:

$$\hat{x}_0 - x_0 = -\frac{\sigma_\tau}{\alpha_\tau} e_\epsilon.$$

Следовательно,

$$\|\hat{x}_0 - x_0\|_2^2 = \frac{\sigma_\tau^2}{\alpha_\tau^2} \|e_\epsilon\|_2^2.$$

Чтобы weighted x_0 -loss совпала с unweighted ϵ -loss, требуется

$$w_{x_0}(\tau) = \frac{\alpha_\tau^2}{\sigma_\tau^2}.$$

Velocity-prediction error

Из

$$\epsilon = \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}}$$

при одном и том же input x_τ :

$$\hat{\epsilon} - \epsilon = \alpha_\tau (\hat{v} - v_\tau^{\text{VP}}).$$

Поэтому

$$\|\hat{v} - v_{\tau}^{\text{VP}}\|_2^2 = \frac{1}{\alpha_{\tau}^2} \|e_{\epsilon}\|_2^2.$$

Для literal equality с unweighted ϵ -loss нужен weight $w_v(\tau) = \alpha_{\tau}^2$.

Три уровня «эквивалентности parameterizations»

Получается полезное разделение.

Уровень 1. Algebraic equivalence targets

При известных $\alpha_{\tau}, \sigma_{\tau}$ и away from singular endpoints можно перевести exact x_0 , ϵ , score target и normalized v -target друг в друга.

Это точное algebraic statement.

Уровень 2. Equivalence ideal predictors

В unrestricted population regression соответствующие conditional means тоже переводятся друг в друга.

Если weight зависит только от τ и positive, pointwise Bayes predictor для каждого (x, τ) не меняется: conditional mean остаётся conditional mean.

Это более сильное statement, но оно всё ещё idealized.

Уровень 3. Equivalence реально обученных models

Вот это уже не следует.

Одна finite-capacity network использует общие parameters для разных noise levels. Если loss weights меняются, меняются gradient magnitudes и компромисс между regions path.

Добавляются optimization dynamics, regularization, finite data, clipping, numerical conditioning и architecture details.

Поэтому две модели, одна trained на plain ϵ -MSE, а вторая на plain x_0 -MSE, вообще говоря, не обязаны после algebraic conversion давать одинаковую learned function.

Algebraic invertibility target не является theorem об одинаковом training result.

Пример

Вернёмся к

$$\alpha_\tau = \sigma_\tau = \frac{1}{\sqrt{2}}, \quad x_0 = 2, \quad \epsilon = -1.$$

Мы уже получили

$$x_\tau = \frac{1}{\sqrt{2}}.$$

Conditional score:

$$s_\tau^{\text{cond}} = -\frac{-1}{1/\sqrt{2}} = \sqrt{2}.$$

Velocity-prediction target:

$$\begin{aligned} v_\tau^{\text{VP}} &= \frac{-1}{\sqrt{2}} - \frac{2}{\sqrt{2}} \\ &= -\frac{3}{\sqrt{2}}. \end{aligned}$$

Восстановим clean sample:

$$\begin{aligned} x_0 &= \alpha_\tau x_\tau - \sigma_\tau v_\tau^{\text{VP}} \\ &= \frac{1}{2} + \frac{3}{2} = 2. \end{aligned}$$

Восстановим noise:

$$\begin{aligned} \epsilon &= \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}} \\ &= \frac{1}{2} - \frac{3}{2} = -1. \end{aligned}$$

Алгебра работает точно.

Но если эти targets использовать с разными unweighted losses по всему interval $\tau \in [0, 1]$, scale errors будет зависеть от τ по-разному.

Где здесь легко ошибиться

«Раз parameterizations invertible, trained models одинаковы»

Нет.

Invertibility говорит про target values, а не про weighting optimization problem.

«Unweighted ϵ -loss и unweighted score loss - одна loss»

Нет.

Они отличаются factor $1/\sigma_\tau^2$.

« x_0 -prediction всегда numerical stable, потому что clean target простой»

Не следует.

Conversion из ϵ в x_0 делит на α_τ . При $\alpha_\tau \rightarrow 0$ это плохо conditioned.

A conversion из ϵ в score делит на σ_τ , что problematic около clean endpoint.

«Diffusion v - это ODE velocity»

Не как definition.

Для normalized schedule conditional transport velocity равна $\dot{\phi}_\tau v_\tau^{\text{VP}}$, а marginal ODE field дополнительно требует conditional averaging.

Что теперь понятно

Четыре parameterizations действительно описывают тесно связанные quantities одного Gaussian path.

Но нужно всегда задавать два дополнительных вопроса:

1. Как именно output преобразуется в нужный physical quantity?
2. Какой weight после этого фактически получает error на каждом noise level?

Без второго вопроса утверждение «objectives equivalent» неполно.

4. Score SDE, probability-flow ODE и связь с Flow Matching

Зачем эта тема здесь

До этого score был probabilistic quantity, а Flow Matching использовал transport velocity.

Теперь нужно соединить эти языки напрямую.

Score-based SDE framework Song et al. показывает две вещи:

1. exact marginal score позволяет записать reverse stochastic dynamics;
2. той же diffusion соответствует deterministic probability-flow ODE с теми же time marginals.

Именно второй result помещает score-based diffusion внутрь знакомого transport language. Работа была опубликована на ICLR 2021.

Интуиция

SDE можно пока воспринимать как infinitesimal update с двумя частями.

Одна часть - обычный drift: $f(x, \tau)d\tau$.

Вторая - small random Gaussian kick: $g(\tau)dW_\tau$.

На маленьком interval $d\tau$ Brownian increment по scale похож на $dW_\tau \sim \mathcal{N}(0, d\tau I)$.

Поэтому forward SDE постепенно и непрерывно добавляет randomness.

Самое содержательное утверждение score-based framework: если известен score текущего noisy marginal, можно математически исправить drift так, чтобы идти process назад.

А если stochastic trajectories не нужны, существует deterministic ODE, которое двигает distributions через те же p_τ .

Понятия, которые вводятся

Нужны SDE, Brownian motion, drift, diffusion coefficient, reverse-time SDE, probability-flow ODE и diffusion path.

Формальное изложение

Forward SDE:

$$dX_\tau = f(X_\tau, \tau)d\tau + g(\tau)dW_\tau.$$

Здесь:

- f - deterministic drift;
- g - noise amplitude;
- W_τ - Brownian motion.

Полную SDE theory, Itô calculus и Fokker-Planck derivation мы не вводим.

Нам нужны results.

Reverse-time SDE

Для exact marginal score $s_\tau(x) = \nabla_x \log p_\tau(x)$ reverse dynamics можно записать как

$$dX_\tau = [f(X_\tau, \tau) - g(\tau)^2 s_\tau(X_\tau)]d\tau + g(\tau)d\bar{W}_\tau,$$

если equation интегрируется от большого τ к малому.

Здесь $\bar{W}$ - reverse-time Brownian motion.

Направление integration принципиально. Если вместо decreasing τ ввести новый increasing reversed clock, signs в записанной equation изменятся. Поэтому reverse-SDE formula нельзя запоминать отдельно от time convention.

Полный derivation этого theorem в конспект не входит.

Probability-flow ODE

Song et al. также получают deterministic dynamics

$$\frac{dx}{d\tau} = f(x, \tau) - \frac{1}{2}g(\tau)^2 s_\tau(x).$$

Это probability-flow ODE.

Ключевое statement:

при exact score probability-flow ODE и соответствующая SDE имеют одинаковые marginal distributions p_τ для каждого time τ .

Но это не означает одинаковые sample trajectories.

SDE stochastic.

Probability-flow ODE deterministic при фиксированном initial state.

Совпадают time marginals, а не pathwise dynamics.

Теперь возвращаем Gaussian path занятия 4

Пусть $X_\tau = \alpha_\tau X_0 + \sigma_\tau \epsilon$.

Conditional velocity получается обычным differentiation:

$$U_\tau = \dot{\alpha}_\tau X_0 + \dot{\sigma}_\tau \epsilon.$$

Никакой SDE theory для этой formula не нужна.

Flow Matching говорит, что corresponding marginal field получается averaging: $v_\tau(x) = \mathbb{E}[U_\tau \mid X_\tau = x]$.

Теперь выразим его через score.

Шаг 1. Выразим X_0 через x и ϵ

При $\alpha_\tau \neq 0$:

$$X_0 = \frac{X_\tau - \sigma_\tau \epsilon}{\alpha_\tau}.$$

Подставляем в U_τ :

$$U_\tau = \frac{\dot{\alpha}_\tau}{\alpha_\tau} X_\tau + \left(\dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau \right) \epsilon.$$

Обозначим только локально для этой derivation

$$\lambda_\tau = \dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau.$$

Тогда

$$U_\tau = \frac{\dot{\alpha}_\tau}{\alpha_\tau} X_\tau + \lambda_\tau \epsilon.$$

Шаг 2. Marginalize hidden noise

Condition on $X_\tau = x$:

$$v_\tau(x) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x + \lambda_\tau \mathbb{E}[\epsilon \mid X_\tau = x].$$

Из denoising identity:

$$\mathbb{E}[\epsilon \mid X_\tau = x] = -\sigma_\tau s_\tau(x).$$

Следовательно,

$$v_\tau(x) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x - \sigma_\tau \lambda_\tau s_\tau(x).$$

Раскрывая λ_τ :

$$v_\tau(x) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x - \sigma_\tau \left(\dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau \right) s_\tau(x).$$

Вот уже прямой bridge:

marginal score $\longleftrightarrow$ marginal transport velocity.

Как здесь появляется probability-flow ODE

Для linear Gaussian diffusion SDE, которая реализует тот же Gaussian probability path, coefficients связаны с α_τ, σ_τ так:

$$f(x, \tau) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x$$

и

$$\frac{1}{2}g(\tau)^2 = \sigma_\tau \left(\dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau \right).$$

Эти relations даём как result, без SDE derivation.

Подставляем в probability-flow ODE:

$$\frac{dx}{d\tau} = f(x, \tau) - \frac{1}{2}g(\tau)^2 s_\tau(x).$$

Получаем ровно

$$\frac{dx}{d\tau} = v_\tau(x).$$

То есть при одном и том же valid Gaussian diffusion path:

probability-flow ODE field = marginal FM field.

Вот в каком точном смысле score-based diffusion и Flow Matching могут оказаться описаниями одного transport.

Flow Matching paper прямо строит framework для general Gaussian probability paths и отмечает diffusion paths как special cases, но при этом допускает и другие paths.

Есть важное условие: expression, определяющее $g(\tau)^2$, должно быть non-negative. Не вся arbitrary interpolation α_τ, σ_τ обязана задавать обычную scalar-noise forward diffusion SDE.

А что с training objectives

Даже если marginal field один и тот же, training losses ещё не обязаны быть одинаковыми.

У нас было

$$U_\tau = \frac{\dot{\alpha}_\tau}{\alpha_\tau} X_\tau + \lambda_\tau \epsilon.$$

Если velocity predictor строится из noise predictor:

$$\hat{U}_\tau = \frac{\dot{\alpha}_\tau}{\alpha_\tau} X_\tau + \lambda_\tau \hat{\epsilon}_\theta,$$

то error:

$$\hat{U}_\tau - U_\tau = \lambda_\tau (\hat{\epsilon}_\theta - \epsilon).$$

Следовательно,

$$\|\hat{U}_\tau - U_\tau\|_2^2 = \lambda_\tau^2 \|\hat{\epsilon}_\theta - \epsilon\|_2^2.$$

Plain FM velocity MSE и plain ϵ -MSE поэтому отличаются time-dependent weight λ_τ^2 .

Если записать

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \left[w_u(\tau) \|\hat{U}_\tau - U_\tau\|_2^2 \right],$$

то после conversion в ϵ coordinates:

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \left[w_u(\tau) \lambda_\tau^2 \|\hat{\epsilon}_\theta - \epsilon\|_2^2 \right].$$

Чтобы она буквально совпала с

$$\mathcal{L}_\epsilon = \mathbb{E} \left[w_\epsilon(\tau) \|\hat{\epsilon}_\theta - \epsilon\|_2^2 \right],$$

нужно

$$w_\epsilon(\tau) = w_u(\tau) \lambda_\tau^2.$$

Иначе objectives different, даже если target information алгебраически одно и то же.

Когда корректно говорить об equivalence diffusion и Flow Matching

Нужно как минимум зафиксировать:

1. **Probability path.** Одни и те же p_τ , то есть один schedule и одна orientation time.
2. **Conditional construction.** Одна и та же Gaussian coupling/noising construction, если сравниваются conditional targets.
3. **Parameterization.** Должно быть явно сказано, какая variable предсказывается и как outputs преобразуются.
4. **Distribution noise levels.** τ должна sampled из согласованной distribution.
5. **Loss weighting.** Time-dependent scale после target conversion нужно либо сохранить, либо осознанно изменить.
6. **Model class и optimization.** Даже совпадающие population objectives не обещают identical parameter tensors и identical finite-training result.

Поэтому корректная формулировка такая:

для фиксированного Gaussian probability path score, noise prediction и corresponding transport velocity содержат преобразуемую информацию; probability-flow ODE задаёт тот же marginal transport, что и marginal Flow Matching field. Но конкретные training objectives совпадают только после согласования parameterization и weighting.

Это существенно слабее фразы «diffusion и flow matching - одно и то же», но зато это точное statement.

Пример: normalized angular path

Пусть

$$\alpha_\tau = \cos \phi_\tau, \quad \sigma_\tau = \sin \phi_\tau.$$

Мы уже получили $U_\tau = \dot{\phi}_\tau v_\tau^{\text{VP}}$.

Значит

$$\|\hat{U}_\tau - U_\tau\|_2^2 = \dot{\phi}_\tau^2 \|\hat{v} - v_\tau^{\text{VP}}\|_2^2.$$

Если angle растёт linearly:

$$\phi_\tau = \frac{\pi\tau}{2},$$

то $\dot{\phi}_\tau = \pi/2$ constant.

Тогда two losses отличаются только global multiplicative constant.

Но если ϕ_τ nonlinear, $\dot{\phi}_\tau^2$ становится time-dependent weighting.

Это хороший маленький пример того, почему statement об equivalence всегда зависит от schedule и clock parameterization.

Где здесь легко ошибиться

«Probability-flow ODE повторяет trajectories reverse SDE»

Нет.

Совпадают marginals p_τ , а не individual paths.

«Score SDE и Flow Matching всегда задают один operator»

Нет.

Flow Matching допускает probability paths, которые не обязаны быть chosen diffusion path.

«Раз marginal fields совпали, losses тоже совпали»

Нет.

После target conversion остаётся time-dependent metric/weight.

«Если ideal population optimum один, weighting вообще не важна»

Для unrestricted function space positive time weighting действительно может сохранять pointwise Bayes optimum.

Но реальная network имеет shared finite parameters и обучается finite optimization procedure. Weighting меняет gradient allocation между noise levels и поэтому обычно меняет learned approximation.

Именно здесь statement об identical training models ломается.

«В reverse SDE можно забыть, в каком направлении идёт τ »

Нет.

Equation с reverse Brownian motion обычно интегрируется от high noise к low noise. При смене time coordinate signs меняются.

Что теперь понятно

Gaussian diffusion path можно описать сразу двумя языками:

density geometry через $s_\tau(x)$,
transport geometry через $v_\tau(x)$.

При соответствующей SDE probability-flow ODE связывает их напрямую.

Поэтому diffusion действительно находится в том же transport landscape, что и Flow Matching.

Но path equivalence, parameterization equivalence, loss equivalence и trained-model equivalence - не одно утверждение.

5. DiT: diffusion objective и backbone - разные оси

Зачем эта тема здесь

После score/SDE formulas легко начать воспринимать diffusion model как конкретную neural architecture.

Исторически image diffusion долго ассоциировалась с U-Net backbone. Но probabilistic construction вообще не требует U-Net.

DiT хорошо отделяет две design axes:

как устроен generative process и какая network approximates required function.

Интуиция

Diffusion construction отвечает на вопросы:

- какой probability path используется;
- какой target предсказывается;
- как sampled noise level;
- каким sampler идём обратно.

Backbone отвечает на другой вопрос:

- какая function class реализует mapping $(x_\tau, \tau, c) \mapsto \text{prediction}$.

Можно заменить CNN/U-Net на Transformer, не меняя смысла Gaussian corruption или score.

И наоборот, один и тот же Transformer-like backbone можно использовать с разными flow/diffusion objectives.

Понятия, которые вводятся

Нужны Diffusion Transformer, latent patch, diffusion backbone и conditioning.

Формальное изложение

Удобно разделять минимум три оси.

Ось 1. Generative objective / path

Например: $x_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ и ϵ -prediction, score prediction или velocity prediction.

Ось 2. Backbone

Например:

- U-Net;
- Transformer;
- linear-attention Transformer;
- hybrid architecture.

Ось 3. Sampler

Например:

- reverse stochastic sampler;
- probability-flow ODE solver;
- конкретный discretization schedule;
- guidance scheme.

Смена одной оси не требует автоматически менять остальные.

Что делает DiT

Peebles и Xie в DiT заменяют распространённый U-Net backbone latent diffusion model на Transformer, работающий с patches latent representation. Работа появилась на arXiv в декабре 2022 года и была опубликована на ICCV 2023.

С точки зрения текущего занятия важен не полный DiT block design, а separation of concerns: **diffusion objective $\neq$ U-Net**.

Transformer получает noisy latent patches, time/noise conditioning и, в conditional setting, дополнительное condition.

Но Gaussian forward corruption от этого не перестаёт быть Gaussian corruption.

Score не становится attention score.

Probability-flow ODE не меняет definition из-за того, что внутри vector field теперь Transformer.

Пример

Представим две systems.

У первой:

- Gaussian diffusion path;
- ϵ -prediction;
- U-Net backbone;
- 30-step sampler.

У второй:

- тот же Gaussian path;
- та же ϵ -objective;
- Transformer backbone;
- тот же sampler.

Изменился backbone.

Не изменились definitions p_T , score и denoising target.

Теперь другой переход:

- оставляем Transformer;
- вместо ϵ -loss используем Flow Matching velocity objective на другом probability path.

Здесь architecture может почти не измениться, зато изменился generative objective.

Поэтому название backbone не определяет generative formulation.

SANA как cross-link с первой половиной курса

SANA полезна здесь ровно как контрпример простой картине «мы выбрали iterative diffusion, дальше архитектура внутри шага уже не важна». SANA использует Linear DiT и отдельно уменьшает число latent tokens через более сильное compression. То есть внутри iterative whole-object generator всё ещё можно оптимизировать sequence mixer, attention cost и memory footprint каждого NFE. Paper появилась на arXiv в 2024 году и была опубликована на ICLR 2025.

В основной transport theory это ничего не добавляет. Это именно architecture/cost example.

Где здесь легко ошибиться

«Diffusion model - это U-Net»

Нет.

U-Net - один possible backbone.

«DiT - новый diffusion objective»

Нет.

Главная идея DiT - Transformer backbone для latent diffusion.

«Раз внутри Transformer, generation становится autoregressive»

Нет.

Backbone architecture и factorization generative process - разные axes.

DiT может обрабатывать latent patches одновременно внутри одного denoising evaluation.

«Более efficient attention автоматически уменьшает NFE»

Нет.

Он может уменьшить стоимость одного evaluation.

NFE определяется sampler и generative dynamics.

Что теперь понятно

Стоимость iterative generator имеет как минимум два independent levers:

сколько **evaluations** × стоимость одного **evaluation**.

Flow/diffusion/sampler влияют на первый и на required function.

Backbone и sequence mixer сильно влияют на второй.

Именно поэтому generative objective и neural architecture полезно анализировать отдельно.

6. Classifier-Free Guidance

Зачем эта тема здесь

После занятия 4 NFE уже нельзя было путать с solver steps.

CFG добавляет ещё один слой.

Даже если sampler просит vector field один раз, guided prediction может требовать **две predictions network**:

- conditional;
- unconditional.

Значит, фраза «30 sampling steps = 30 forward passes» уже может быть неверной даже для Euler-like sampler.

Интуиция

Conditional model хочет следовать condition c .

Unconditional model описывает prediction без этого condition.

CFG смотрит на difference: $\hat{\epsilon}_c - \hat{\epsilon}_\emptyset$.

Грубо говоря, это direction, в котором prediction меняется из-за condition.

Guidance extrapolates в этом direction.

Понятия, которые вводятся

Нужны Classifier-Free Guidance, conditional prediction, unconditional prediction, guidance scale, guidance branch и effective computation.

Формальное изложение

Для ϵ -parameterization одна стандартная convention:

$$\hat{\epsilon}_{\text{cfg}} = \hat{\epsilon}_{\emptyset} + w(\hat{\epsilon}_c - \hat{\epsilon}_{\emptyset}).$$

Здесь:

- $\hat{\epsilon}_c$ - conditional prediction;
- $\hat{\epsilon}_{\emptyset}$ - unconditional prediction;
- w - guidance scale.

При этой convention: $w = 0$ даёт unconditional prediction, $w = 1$ даёт обычную conditional prediction, а $w > 1$ extrapolates дальше conditional prediction.

В libraries встречаются и другие parameterizations guidance scale, поэтому значение hyperparameter нужно интерпретировать вместе с formula, а не только по имени `guidance_scale`.

Original classifier-free guidance construction совместно обучает conditional и unconditional behaviour без отдельного classifier; short version появилась на NeurIPS workshop 2021, расширенный arXiv preprint - в 2022 году.

Откуда берётся дополнительная стоимость

Чтобы вычислить $\hat{\epsilon}_{\text{cfg}}$, нужны обе quantities

$$\hat{\epsilon}_c, \quad \hat{\epsilon}_{\emptyset}.$$

Обычная implementation может получить их двумя способами.

Вариант 1. Два отдельных forward passes

Сначала: $\hat{\epsilon}_c = f_{\theta}(x_{\tau}, \tau, c)$.

Потом:

$$\hat{\epsilon}_{\emptyset} = f_{\theta}(x_{\tau}, \tau, \emptyset).$$

Один request sampler к guided field породил два network forwards.

Вариант 2. Один batched forward

Можно concatenated два conditions по batch dimension:

$$\begin{pmatrix} x_T, c \\ x_T, \emptyset \end{pmatrix}.$$

Тогда на Python level может быть один вызов `model(...)`.

Но batch в этом call вдвое больше.

То есть **один literal function call не означает вычислительную стоимость одной prediction.**

Это особенно важно при benchmarking.

Пример: 25 Euler steps

Пусть sampler использует 25 Euler steps.

Без guidance Euler требует одну field evaluation на step.

Наивно: $N_{FE} = 25$.

С vanilla CFG нужны 25 conditional и 25 unconditional predictions.

Если выполнять их sequentially: $25 + 25 = 50$ network forward computations.

Если batching branches вместе, Python может увидеть всего 25 calls, но каждый call обрабатывает batch вдвое больше.

Предположим, backbone получает 1024 latent patches на sample.

Без CFG network обрабатывает $25 \cdot 1024 = 25600$ patch positions.

С vanilla CFG: $50 \cdot 1024 = 51200$.

Количество sampler steps осталось 25.

Количество literal Python calls может остаться 25.

Но processed workload уже другой.

Почему нельзя просто сказать «CFG ровно удваивает latency»

Compute двух branches действительно не становится бесплатным.

Но wall-clock latency зависит от implementation:

- два sequential forwards;
- batch fusion;
- GPU occupancy;
- memory bandwidth;
- kernel shapes;
- batch size;
- compiler;
- возможный reuse части computation.

Поэтому правильная формулировка:

vanilla CFG требует conditional и unconditional computations и примерно удваивает single-branch model work, но не гарантирует ровно $2\times$ wall-clock latency.

Peak memory тоже зависит от implementation.

Два sequential calls могут reuse один workspace.

Batched conditional/unconditional call может потребовать больше simultaneous activation memory.

Где здесь легко ошибиться

«Один sampler step - один network evaluation»

Даже у Euler это ломается из-за guidance branches.

У higher-order solver дополнительно может быть несколько field evaluations на macro-step.

«Один Python call - одна единица работы»

Нет.

Внутри call batch может иметь два CFG branches.

«CFG обязательно означает две разные networks»

Нет.

Обычно conditional и unconditional behaviour реализует одна network с condition dropout / null condition.

«CFG ровно удваивает peak memory»

Не обязательно.

Sequential two-pass и batched two-branch implementation имеют разные memory profiles.

«Если calls fused, вторая prediction бесплатна»

Нет.

Она всё равно требует обработки дополнительного batch element, хотя hardware utilization может стать лучше.

Что теперь понятно

Для guided iterative generator слово «шаг» уже слишком неоднозначно.

Нужно отдельно знать:

- сколько macro-steps делает sampler;
- сколько field evaluations требует solver;
- сколько conditional branches требуется на evaluation;
- сколько elements обрабатывает каждый network forward;
- как эти computations batched.

И только после этого имеет смысл говорить про latency и throughput.

7. Метрики эффективности: что именно считать вместо «числа проходов»

Зачем эта тема здесь

В первой половине курса уже был введён cost vector

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

В нём memory, traffic, arithmetic, sequential depth, NFE и concurrency специально были разделены.

Для iterative generation это разделение становится ещё важнее.

Фраза «модель делает 50 проходов» почти ничего не говорит, пока непонятно:

- что называется проходом;
- сколько данных проходит через network;
- насколько дорог один evaluation;
- сколько branches есть внутри;
- какая peak memory;
- какая реальная latency.

Интуиция

Сравним два imaginary generators.

У первого 20 evaluations огромного Transformer над 4096 latent patches.

У второго 40 evaluations маленькой network над 256 patches.

По NFE первый вроде бы «в два раза дешевле».

По FLOPs или latency всё может оказаться наоборот.

И ещё один independent вопрос: оба algorithms могут reuse workspace между sequential steps, поэтому 40 steps не означают в 40 раз больше peak memory.

Понятия, которые вводятся

Здесь нужны sampler steps, NFE, processed elements, latency, throughput и peak memory.

Формальное изложение

1. Sampler steps

Пусть numerical sampler делает K macro-steps.

Это число продвижений time variable.

Оно полезно, но не определяет количество network evaluations.

Euler обычно использует одну evaluation на step.

Classical RK4 - четыре.

Adaptive solver может иметь rejected attempts.

CFG добавляет внутренние branches.

2. NFE

N_{FE} отвечает на вопрос:

сколько раз numerical algorithm запросил learned function?

Но при guidance нужно явно указать convention.

Если guided field внутри сам вычисляет conditional и unconditional branch, можно:

- считать один evaluation guided field;
- или считать две single-branch network evaluations.

Для hardware comparison одного NFE уже недостаточно. Поэтому вместе с NFE нужно давать branch/batch convention.

3. Число обработанных elements

Пусть j индексирует фактические branch computations.

Пусть computation j обрабатывает batch B_j и L_j latent patches, pixels или tokens на sample.

Тогда useful workload proxy:

$$N_{\text{processed}} = \sum_j B_j L_j.$$

Для image DiT L_j обычно означает число latent patches.

Для text models на следующем занятии эта quantity станет числом реально обработанных token positions.

Такой счётчик всё ещё не равен FLOPs, поскольку разные architectures имеют разную стоимость per element. Но он уже намного информативнее слова «проход».

4. Latency

Latency - wall-clock time получения результата.

Её нельзя вывести только из NFE.

Она зависит от:

- architecture;
- tensor shapes;
- precision;
- kernels;
- memory traffic;
- batching;
- guidance implementation;
- solver overhead;
- hardware.

Поэтому latency - **измеряемая system metric**, а не synonym theoretical step count.

5. Throughput

Throughput отвечает на другой вопрос:

сколько samples система завершает за единицу времени при определённой concurrency?

Например,

$$\text{throughput} = \frac{N_{\text{samples}}}{\Delta T}.$$

Latency и throughput могут двигаться в разные стороны.

Большой batch часто повышает GPU utilization и throughput, но individual request может ждать дольше.

Поэтому benchmark без batch/concurrency setting неполон.

6. Peak memory

Peak memory:

$$M_{\text{peak}} = \max_{\text{execution time}} M_{\text{allocated}}.$$

Это максимум simultaneous memory footprint.

Не сумма memory allocations всех sequential steps.

Если 50 diffusion steps выполняются последовательно и workspace reuse, activations step 1 не обязаны жить одновременно с activations step 50.

Поэтому нельзя умножать memory одного forward на NFE.

Но из этого не следует, что diffusion memory маленькая.

Почему «диффузия требует мало памяти» неверно

Standard image diffusion часто не хранит растущий autoregressive KV-cache между denoising steps.

Это действительно может уменьшать одну component persistent state.

Но полный memory footprint включает как минимум:

$$M_{\text{weights}} + M_{\text{persistent}} + M_{\text{workspace}} + M_{\text{runtime}}.$$

У большого DiT:

- weights могут быть большими;

- full attention over latent patches может иметь substantial workspace;
- activations зависят от sequence length latent patches;
- CFG batching может увеличить simultaneous workload;
- conditioning networks и VAE тоже используют memory.

То есть отсутствие growing KV-cache ничего само по себе не говорит о M_{peak} .

Правильный statement намного уже:

конкретная diffusion architecture может не иметь autoregressive persistent KV-cache, но её weights и per-evaluation workspace могут быть большими.

И наоборот, большое NFE увеличивает total computation и latency, но не обязано линейно увеличивать peak memory.

Worked example: почему одной цифры «30 проходов» недостаточно

Пусть система использует:

- 30 Euler steps;
- vanilla CFG;
- batch size 1;
- 1024 latent patches;
- DiT backbone.

Sampler requests: **30** guided fields.

Для каждого нужны две branches.

Single-branch-equivalent computations: **$30 \cdot 2 = 60$** .

Processed latent positions: **$60 \cdot 1024 = 61440$** .

Теперь implementation A делает два sequential model calls.

Implementation B concatenates branches и делает один batch-2 call на step.

У A:

- 60 literal model calls;
- smaller per-call batch;

- потенциально ниже simultaneous activation footprint.

У В:

- 30 literal model calls;
- batch вдвое больше;
- те же 60 branch predictions;
- потенциально выше GPU utilization;
- потенциально выше peak workspace.

Ни «30», ни «60» сами по себе не дают latency.

Её нужно измерять.

Peak memory тоже нельзя получить умножением на 30 или 60.

Где здесь легко ошибиться

«Меньше NFE означает меньше latency»

При одинаковом backbone и похожей implementation обычно это полезный signal, но как общее theorem неверно.

Один evaluation может сильно отличаться по cost.

«Больше sampler steps означает больше peak memory»

Нет.

Sequential computations могут reuse memory.

«Throughput - это обратная latency»

Только в очень специальных single-request settings.

При batching и concurrency это разные metrics.

«Processed elements полностью определяют FLOPs»

Нет.

Attention, convolution, MoE и linear attention могут иметь разную arithmetic cost при одном числе elements.

«Раз нет KV-cache, memory problem решена»

Нет.

Persistent cache - только одна component cost vector.

«CFG удвоил branches, значит всё ровно 2×»

Не обязательно для latency и memory.

Но дополнительная model computation реальна и должна быть учтена.

Что теперь понятно

Вместо «числа проходов» у iterative generator нужно минимум сообщать:

- sampler steps;
- solver/model NFE с явной convention;
- guidance branches;
- processed elements;
- latency;
- throughput и concurrency;
- peak memory.

Для серьёзного systems comparison к этому добавляются FLOPs и HBM traffic.

Это продолжает cost-vector framework первого занятия, а не вводит новую метафору «память против проходов».

Главная схема занятия

Forward diffusion construction:

$$\begin{aligned}
X_0 &\sim p_{\text{data}}, & \epsilon &\sim \mathcal{N}(0, I) \\
&\downarrow \\
X_\tau &= \alpha_\tau X_0 + \sigma_\tau \epsilon \\
&\downarrow \\
p(x_\tau | x_0) &= \mathcal{N}(\alpha_\tau x_0, \sigma_\tau^2 I).
\end{aligned}$$

Из conditional Gaussian получаем training score target:

$$\begin{aligned}
&\nabla_{x_\tau} \log p(x_\tau | x_0) \\
&= -\frac{x_\tau - \alpha_\tau x_0}{\sigma_\tau^2} \\
&= -\frac{\epsilon}{\sigma_\tau}.
\end{aligned}$$

Denoising identity превращает conditional target в marginal quantity:

$$s_\tau(x) = -\frac{1}{\sigma_\tau} \mathbb{E}[\epsilon | X_\tau = x].$$

Дальше одна и та же Gaussian construction допускает несколько output coordinates:

$$\epsilon \longleftrightarrow x_0 \longleftrightarrow s_\tau \longleftrightarrow v^{\text{VP}}.$$

Но corresponding unweighted MSE losses имеют разные time-dependent scales.

Для Gaussian path conditional transport velocity:

$$U_\tau = \dot{\alpha}_\tau X_0 + \dot{\sigma}_\tau \epsilon.$$

После marginalization:

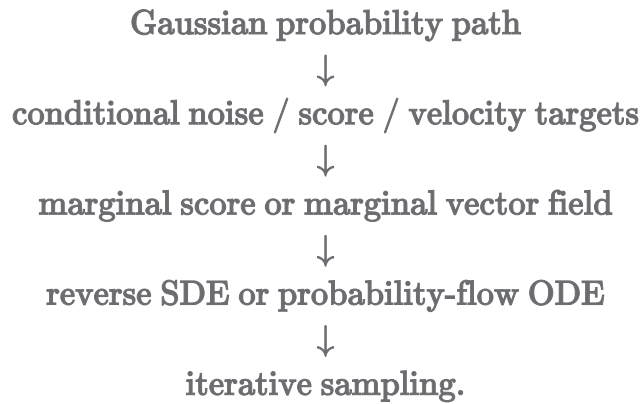
$$v_\tau(x) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x - \sigma_\tau \lambda_\tau s_\tau(x),$$

где

$$\lambda_\tau = \dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau.$$

Для соответствующей diffusion SDE это ровно probability-flow ODE field.

Получается общий transport landscape:



И две главные оговорки занятия:

$$\text{score} = \nabla \log p, \quad \text{не } \nabla p$$

и

$$\text{algebraic target equivalence} \not\Rightarrow \text{identical training objective.}$$

Вопросы для самопроверки

1. В каком направлении идёт forward noising process?

Ответ.

От clean data к noise.

При convention этого занятия $\tau = 0$ соответствует clean side, а большой τ - noisy side.

Generation идёт в обратную сторону.

2. Почему при $X_\tau = \alpha_\tau x_0 + \sigma_\tau \epsilon$ conditional variance равна $\sigma_\tau^2 I$?

Ответ.

x_0 при conditioning фиксирован, поэтому randomness идёт только от $\sigma_\tau \epsilon$.

У standard Gaussian $\text{Cov}(\epsilon) = I$.

Scaling vector на σ_τ умножает covariance на σ_τ^2 .

3. Как выглядит $p(x_\tau \mid x_0)$?

Ответ.

$$p(x_\tau \mid x_0) = \mathcal{N}(\alpha_\tau x_0, \sigma_\tau^2 I).$$

4. Что такое score?

Ответ. $s_\tau(x) = \nabla_x \log p_\tau(x)$.

Это gradient log-density noisy marginal.

5. Почему $\nabla p_\tau(x)$ не является score?

Ответ.

Потому что

$$\nabla p_\tau(x) = p_\tau(x) \nabla \log p_\tau(x).$$

Gradient density содержит дополнительный density scale.

Score измеряет relative local change и invariant к multiplicative normalization constant.

6. Как вывести conditional Gaussian score?

Ответ.

Log conditional density содержит term

$$-\frac{1}{2\sigma_\tau^2} \|x_\tau - \alpha_\tau x_0\|_2^2.$$

Gradient quadratic norm равен $2(x_\tau - \alpha_\tau x_0)$.

Поэтому

$$\nabla_{x_\tau} \log p(x_\tau | x_0) = -\frac{x_\tau - \alpha_\tau x_0}{\sigma_\tau^2}.$$

7. Почему conditional score равен $-\epsilon/\sigma_\tau$?

Ответ.

Потому что corruption construction даёт $x_\tau - \alpha_\tau x_0 = \sigma_\tau \epsilon$.

Подстановка сокращает один factor σ_τ .

8. Что утверждает denoising identity?

Ответ.

Marginal score является posterior average conditional scores:

$$s_\tau(x) = \mathbb{E} [\nabla_x \log p(X_\tau = x | X_0) | X_\tau = x].$$

Для Gaussian corruption:

$$s_\tau(x) = -\frac{1}{\sigma_\tau} \mathbb{E}[\epsilon | X_\tau = x].$$

9. Почему MSE ϵ -prediction не обязана восстанавливать конкретный sampled noise?

Ответ.

Потому что один noisy state может соответствовать нескольким clean/noise pairs.

MSE predictor возвращает conditional expectation noise при observed (x_τ, τ) .

10. Как получить score из ϵ -prediction?

Ответ.

$$\hat{s}_\tau(x_\tau) = -\frac{\hat{\epsilon}_\theta(x_\tau, \tau)}{\sigma_\tau}.$$

11. Как получить x_0 из ϵ -prediction?

Ответ.

При $\alpha_\tau \neq 0$:

$$\hat{x}_0 = \frac{x_\tau - \sigma_\tau \hat{\epsilon}_\theta}{\alpha_\tau}.$$

12. Как определяется diffusion velocity-prediction target?

Ответ.

Для normalized schedule:

$$v_\tau^{\text{VP}} = \alpha_\tau \epsilon - \sigma_\tau x_0.$$

13. Как восстановить x_0 и ϵ из v_τ^{VP} ?

Ответ.

Если $\alpha_\tau^2 + \sigma_\tau^2 = 1$:

$$\begin{aligned} x_0 &= \alpha_\tau x_\tau - \sigma_\tau v_\tau^{\text{VP}}, \\ \epsilon &= \sigma_\tau x_\tau + \alpha_\tau v_\tau^{\text{VP}}. \end{aligned}$$

14. Почему v_τ^{VP} не нужно смешивать с $v_\tau(x)$ из Flow Matching?

Ответ.

v_τ^{VP} - pair-specific diffusion target, построенный из hidden x_0 и ϵ .

$v_\tau(x)$ - marginal vector field, function только текущего state и time.

Даже conditional transport velocity равна v_τ^{VP} только при специальной time parameterization.

15. Какая связь между v -prediction и conditional velocity для normalized angular schedule?

Ответ.

Если

$$\alpha_\tau = \cos \phi_\tau, \quad \sigma_\tau = \sin \phi_\tau,$$

то $U_\tau = \dot{\phi}_\tau v_\tau^{\text{VP}}$.

16. Почему unweighted score loss не равна unweighted ϵ -loss?

Ответ.

Errors связаны:

$$\|\hat{s} - s^{\text{cond}}\|_2^2 = \frac{1}{\sigma_\tau^2} \|\hat{\epsilon} - \epsilon\|_2^2.$$

Значит score loss иначе weights noise levels.

17. Какой weight score loss делает её literal equivalent unweighted ϵ -loss?

Ответ. $w_s(\tau) = \sigma_\tau^2$.

18. Как unweighted x_0 -loss weights ϵ -error?

Ответ.

$$\|\hat{x}_0 - x_0\|_2^2 = \frac{\sigma_\tau^2}{\alpha_\tau^2} \|\hat{\epsilon} - \epsilon\|_2^2.$$

19. Что такое forward SDE на уровне этого курса?

Ответ.

Continuous stochastic process $dX_\tau = f(X_\tau, \tau)d\tau + g(\tau)dW_\tau$, где f задаёт deterministic drift, а Brownian term постепенно добавляет randomness.

Itô calculus для использования следующих results не требуется.

20. Чем reverse-time SDE отличается от probability-flow ODE?

Ответ.

Reverse-time SDE остаётся stochastic и содержит Brownian noise.

Probability-flow ODE deterministic:

$$\frac{dx}{d\tau} = f(x, \tau) - \frac{1}{2}g(\tau)^2 s_\tau(x).$$

При exact score они согласованы по marginal distributions.

21. Что именно одинаково у SDE и probability-flow ODE?

Ответ.

Time marginals p_τ .

Не individual trajectories и не полный joint law paths.

22. Какова conditional velocity Gaussian path?

Ответ.

Для $X_\tau = \alpha_\tau X_0 + \sigma_\tau \epsilon$ обычное differentiation даёт

$$U_\tau = \dot{\alpha}_\tau X_0 + \dot{\sigma}_\tau \epsilon.$$

23. Как marginal FM field связан со score?

Ответ.

При $\alpha_\tau \neq 0$:

$$v_\tau(x) = \frac{\dot{\alpha}_\tau}{\alpha_\tau} x - \sigma_\tau \left(\dot{\sigma}_\tau - \frac{\dot{\alpha}_\tau}{\alpha_\tau} \sigma_\tau \right) s_\tau(x).$$

24. При каких условиях корректно говорить, что diffusion и Flow Matching описывают один transport?

Ответ.

Нужно как минимум зафиксировать один probability path и time convention.

Для соответствующего Gaussian diffusion path exact score через probability-flow ODE задаёт тот же marginal field, который получается marginalization conditional FM velocities.

Для equivalence training objectives дополнительно нужно согласовать target parameterization, distribution τ и loss weights.

25. Почему одинаковый ideal field не гарантирует одинаковую trained network?

Ответ.

Finite-capacity model делит parameters между всеми noise levels.

Разные weights меняют gradient allocation и approximation compromise.

Добавляются optimization, finite data, numerical conditioning и regularization.

26. Что DiT меняет относительно U-Net diffusion model?

Ответ.

Backbone.

DiT использует Transformer над latent patches.

Сам факт Gaussian corruption, score definition и diffusion objective из этого автоматически не меняется.

27. Почему один CFG sampler step может требовать две predictions?

Ответ.

Потому что нужны conditional и unconditional outputs:

$$\hat{\epsilon}_c, \quad \hat{\epsilon}_\emptyset.$$

Vanilla guidance комбинирует оба.

28. Если conditional и unconditional branches concatenated в один batch, стало ли работы вдвое меньше?

Ответ.

Нет.

Literal model call может быть один, но он обрабатывает два batch elements вместо одного.

GPU utilization может улучшиться, но вторая branch не исчезла.

29. Чем sampler steps отличаются от NFE?

Ответ.

Sampler step - macro-step numerical algorithm.

NFE - evaluations learned function.

Higher-order solver может иметь несколько NFE на step.

30. Почему NFE недостаточно для оценки cost?

Ответ.

Evaluations могут отличаться по architecture, sequence/patch length, batch, guidance branches, precision и hardware efficiency.

Поэтому дополнительно нужны processed elements, latency, throughput, memory и другие coordinates cost vector.

31. Почему много diffusion steps не означает proportionally большую peak memory?

Ответ.

Steps выполняются sequentially и обычно reuse workspace.

Peak memory определяется maximum simultaneously live tensors, а не суммой allocations всех steps.

32. Почему из отсутствия autoregressive KV-cache не следует «diffusion мало ест память»?

Ответ.

Остаются weights, current full object, attention workspace, activations, conditioning и runtime buffers.

Большой DiT может иметь substantial peak memory даже без growing KV-cache.

Что читать дальше

Jonathan Ho, Ajay Jain, Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020.

[arXiv:2006.11239](#) - Читать sections про forward process и closed-form $q(x_\ell | x_0)$. Для текущей линии особенно важна связь Gaussian corruption с noise prediction. Полный variational derivation пока можно пропустить.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, Ben Poole. *Score-Based Generative Modeling through Stochastic Differential Equations*. ICLR 2021.

[arXiv:2011.13456](#) - Читать ради общей SDE formulation, reverse-time dynamics и probability-flow ODE. Главный conceptual result для курса - stochastic и deterministic dynamics могут иметь один family marginals, а обе используют marginal score.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le. *Flow Matching for Generative Modeling*. arXiv v1 2022, ICLR 2023.

[arXiv:2210.02747](#) - Вернуться к sections про Gaussian probability paths. После этого занятия paper полезно перечитать второй раз: теперь diffusion path можно увидеть как частный path внутри уже знакомого FM framework, а не как отдельную вселенную.

William Peebles, Saining Xie. *Scalable Diffusion Models with Transformers*. arXiv 2022, ICCV 2023.

[arXiv:2212.09748](#) - Читать ради separation diffusion objective и backbone. Подробности всех DiT blocks для текущего занятия вторичны; важнее увидеть, что U-Net заменяется Transformer, а probabilistic diffusion construction остаётся отдельной axis.

Jonathan Ho, Tim Salimans. *Classifier-Free Diffusion Guidance*. NeurIPS workshop 2021; расширенная arXiv version 2022.

[arXiv:2207.12598](#) - Читать ради conditional/unconditional construction и самой idea linear guidance combination. С точки зрения этого курса дополнительно считать стоимость двух branches.

Enze Xie et al. *SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers*. arXiv 2024, ICLR 2025.

[arXiv:2410.10629](#) - Optional reading. Полезна как связь с первой половиной курса: iterative generator не отменяет architecture-level optimization attention и representation length.

Переход к занятию 6

Всё это занятие опиралось на continuous Euclidean state: $x \in \mathbb{R}^d$.

Поэтому score $\nabla_x \log p_\tau(x)$ имеет обычный смысл gradient по coordinates x .

Для token id ситуация другая.

Token id - category label, а не meaningful Euclidean coordinate. Перестановка номеров vocabulary не должна менять language model, тогда как Gaussian noise над самими ids такую искусственную geometry немедленно создаёт.

Поэтому на следующем занятии Gaussian score не будет механически перенесён на текст. Сначала нужен отдельный discrete bridge: masked/absorbing corruption, затем discrete diffusion / discrete flow language. Именно так устроена программа занятия 6.

Что этот конспект добавляет к контракту нотации

Все соглашения занятия 4 сохраняются.

t остаётся token position.

Continuous noise / transport time: $\tau \in [0, 1]$.

Для discrete diffusion step внутри исторических DDPM formulas в этом курсе используем ℓ , чтобы не переопределять t .

Есть одна осознанная diffusion-specific convention: x_0 в Gaussian corruption formulas означает clean data sample. Поэтому forward noising идёт при increasing τ от data к noise, а generative sampling - обратно. При сопоставлении с orientation занятия 4 direction нужно учитывать явно.

Standard Gaussian noise: $\epsilon \sim \mathcal{N}(0, I)$.

Signal и noise coefficients: α_τ , σ_τ .

Они являются functions continuous time и не переопределяют token-indexed gates из первой половины курса.

Gaussian corruption: $X_\tau = \alpha_\tau X_0 + \sigma_\tau \epsilon$.

Conditional corruption kernel: $p(x_\tau | x_0)$.

Noisy marginal: $p_\tau(x)$.

Marginal score:

$$s_\tau(x) = \nabla_x \log p_\tau(x).$$

Conditional score:

$$s_\tau^{\text{cond}}(x_\tau, x_0) = \nabla_{x_\tau} \log p(x_\tau | x_0).$$

Для Gaussian corruption:

$$s_\tau^{\text{cond}}(x_\tau, x_0) = -\frac{\epsilon}{\sigma_\tau}.$$

Conditional transport velocity сохраняет notation занятия 4:

$$U_\tau = \dot{\alpha}_\tau X_0 + \dot{\sigma}_\tau \epsilon.$$

Marginal ODE vector field также сохраняет notation: $v_\tau(x) = \mathbb{E}[U_\tau | X_\tau = x]$.

Diffusion velocity-prediction target обозначается отдельно:

$$v_\tau^{\text{VP}} = \alpha_\tau \epsilon - \sigma_\tau X_0.$$

v_{τ}^{VP} не является по definition тем же объектом, что $v_{\tau}(x)$.

Forward SDE notation: $dX_{\tau} = f(X_{\tau}, \tau)d\tau + g(\tau)dW_{\tau}$.

f - drift, g - diffusion amplitude, W_{τ} - Brownian motion.

Loss weight на noise level обозначается generic function $w(\tau)$.

N_{FE} сохраняет смысл model/function evaluations, но начиная с этого занятия NFE при guidance нельзя сообщать без convention о conditional/unconditional branches и batching.

Для modality-independent cost дополнительно полезно считать число реально обработанных elements:

$N_{\text{processed}}$.

Для text это будут token positions, для DiT - latent patches.

Главное содержательное соглашение занятия:

probability path, score field, target parameterization, training objective, neural backbone, sampler и hardware cost являются разными уровнями описания. Алгебраическая связь между двумя уровнями не разрешает переносить на остальные свойства без отдельного аргумента.